

Technical Report

Technical Report – Kubernetes Overview

(Prepared for the user's request: "what is Kubernetes and save the information in a PDF file")

1. Executive Summary

Kubernetes (k8s) is an open-source platform that automates the deployment, scaling, and operation of containerized applications. Originally developed by Google and now governed by the Cloud Native Computing Foundation (CNCF), it provides a declarative, highly available, and portable environment for running microservices at scale.

This report consolidates the key concepts, architectural components, operational requirements, benefits, and common challenges of Kubernetes. It also includes a ready-to-run Python script that converts the report into a PDF file, enabling quick distribution or archival.

2. Research Findings

Concept	What it means	Why it matters
---------	---------------	----------------

-----	-----	-----
-------	-------	-------

Cluster	A set of machines (physical or virtual) that run Kubernetes.	Provides the pool of resources (CPU, memory, storage) that workloads consume.
----------------	--	---

Node	A single machine in the cluster. Each node runs the kubelet agent and the container runtime (Docker, containerd, etc.).	Nodes host the containers that make up your applications.
-------------	--	---

Pod	The smallest deployable unit – one or more tightly coupled containers that share the same network namespace and storage.	Pods are the atomic unit of scheduling and scaling.
------------	--	---

Deployment	Declarative object that manages a set of replica Pods, ensuring the desired number of copies are running.	Enables rolling updates, rollbacks, and declarative configuration.
-------------------	---	--

Service	Abstraction that defines a logical set of Pods and a policy to access them (e.g., ClusterIP, NodePort, LoadBalancer).	Provides stable networking and load balancing for Pods.
----------------	---	---

Namespace	Logical partition of cluster resources.	Allows multi-tenant or environment isolation (dev, test, prod).
------------------	---	---

Ingress	API object that manages external access to services, typically via HTTP/HTTPS.	Handles routing, TLS termination, and URL path mapping.
----------------	--	---

ConfigMap / Secret	Key-value stores for configuration data and sensitive information.	Keeps configuration separate from code.
---------------------------	--	---

Horizontal Pod Autoscaler (HPA)	Automatically scales the number of Pods based on metrics (CPU, memory, custom).	Keeps applications responsive under varying load.
--	---	---

| ****Persistent Volumes (PV) / Persistent Volume Claims (PVC)**** | Decouples storage from Pods, enabling stateful workloads. | Allows data persistence across Pod restarts. |

Key Requirements to Run Kubernetes

| Requirement | Typical Tools / Versions | Notes |

|-----|-----|-----|

| ****Container Runtime**** | Docker (v20.10+), containerd, CRI-O | Must support the Container Runtime Interface (CRI). |

| ****Cluster Management**** | kubeadm, kops, minikube, kind, managed services (EKS, GKE, AKS) | Choose based on environment (on-prem, cloud, local dev). |

| ****Control Plane**** | kube-apiserver, kube-controller-manager, kube-scheduler | Runs on master nodes; requires etcd for state storage. |

| ****Networking**** | Calico, Flannel, Weave Net, Cilium | Provides pod-to-pod networking and network policies. |

| ****Storage**** | CSI drivers (e.g., Rook, Ceph, AWS EBS, GCE PD) | For dynamic provisioning of PVs. |

| ****Authentication / Authorization**** | RBAC, OIDC, ServiceAccounts | Controls who can do what in the cluster. |

| ****Monitoring / Logging**** | Prometheus, Grafana, ELK stack, Loki | Essential for observability. |

| ****CI/CD**** | ArgoCD, Flux, Jenkins, GitLab CI | Automates deployments to the cluster. |

Benefits of Using Kubernetes

| Benefit | Explanation |

|-----|-----|

| ****Scalability**** | Horizontal scaling of Pods with HPA and cluster autoscaling. |

| ****High Availability**** | Self-healing: failed Pods are automatically replaced; multiple replicas keep services online. |

| ****Portability**** | Same configuration runs on any cloud or on-prem infrastructure. |

| ****Declarative Management**** | Desired state is defined in YAML; Kubernetes reconciles automatically. |

| ****Microservice Friendly**** | Supports fine-grained services, independent deployments, and canary releases. |

| ****Ecosystem**** | Rich ecosystem of add-ons (Istio, Knative, Helm, Kustomize). |

| ****Cost Efficiency**** | Optimizes resource usage via scheduling and autoscaling. |

Common Challenges

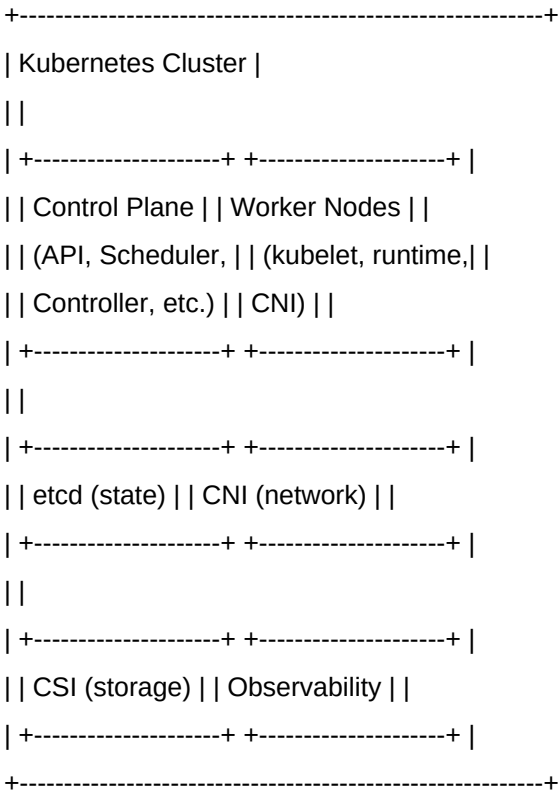
| Challenge | Mitigation |

|-----|-----|

Learning Curve	Start with managed services (EKS, GKE, AKS) or local tools (minikube, kind).
Operational Overhead	Use managed services or operator patterns to reduce day-to-day toil.
Complex Networking	Choose a well-documented CNI plugin; use network policies carefully.
Security	Enforce RBAC, use PodSecurityPolicies or OPA Gatekeeper, keep images signed.
Stateful Workloads	Use CSI drivers and proper backup strategies.
Resource Fragmentation	Use node selectors, taints, and tolerations to control placement.
Cost Management	Monitor cluster usage; delete unused resources; enable autoscaling.

3. Technical Architecture

...



...

- **Control Plane** – the brain: `kube-apiserver`, `kube-controller-manager`, `kube-scheduler`, and `etcd`.
- **Worker Nodes** – run the workloads: `kubelet`, container runtime (`containerd`, `CRI-O`, or Docker), and the network plugin (Calico, Flannel, Cilium, etc.).
- **Add-ons** – networking (CNI), storage (CSI), monitoring (Prometheus), logging (ELK/Loki), and security (OPA, Gatekeeper).

4. Implementation Steps

- **Choose an Installation Path**
- **Local Development** – `minikube`, `kind`, or `k3s`.
- **CI/CD** – `ArgoCD` or `Flux` for GitOps.
- **Managed Cloud** – `EKS`, `GKE`, or `AKS`.
- **OnPrem** – `kubeadm`, `kops`, or `Rancher`.

- **Provision the Cluster**

```
```bash
```

### Example: minikube

```
minikube start --cpus 4 --memory 8192
```

```
```
```

- **Configure Networking**
- Install a CNI plugin (e.g., Calico).
- Verify pod networking: `kubectl get pods -A`.
- **Set Up Storage**
- Deploy a CSI driver (e.g., RookCeph, AWS EBS CSI).
- Create a `StorageClass` for dynamic provisioning.
- **Deploy Core Components**
- Create `Namespace` for isolation.
- Deploy `Deployment`, `Service`, and `Ingress` objects.
- Apply `ConfigMap`/`Secret` for configuration.
- **Enable Autoscaling**
- Deploy HPA: `kubectl autoscale deployment <name> --cpu-percent=50 --min=1 --max=10`.
- **Add Observability**
- Install Prometheus and Grafana (Helm chart: `kube